Ders 18 Çalışma Özeti

Ders 18 Çalışma Özeti

Genel Konular

- Diziler üzerindeki temel işlemler (basic operations on array) (ekrandan, 00:36:30)
  - Arama (search): istenen bir bilginin dizide olup olmadığına bakılır.
  - Ekleme (insert): diziye yeni bir eleman eklenir.
  - Silme (delete): diziden bir eleman çıkarılır.
  - Güncelleme (update): bir elemanın değeri değiştirilir.
  - Her problem bu dört temel işlemi belli oranda içerir. Gerçek hayattan örnekler: yurttan ayrılan öğrencinin kaydını silmek, yanlış toplanan bir notu düzeltmek, alışveriş sepetine ürün eklemek, sepetten ürün çıkarmak ya da ürün sayısını güncellemek.
- `"A"` ile `'A'` farkı (öğrenci sorusu, 00:14:25)
  - Çift tırnak (`"A"`) bir karakter grubudur: `A` ve onu izleyen null karakterini içerir. Karakter dizisinde saklanır.
  - Tek tırnak (`'A'`) tek bir karakter sabitidir. Karakter değişkeninde saklanır.
- Sırasız dizide arama (unsorted array)
  - Akış diyagramı: N ve `Dizi[N]` okunur, aranan X okunur, `I = 1`.
  - While: `I <= N AND Dizi[I]` ≠ `X` olduğu sürece `I = I + 1`.
  - Döngüden sonra `I <= N` ise I (bulunan konum) yazdırılır, değilse “Eleman bulunamadı” yazdırılır.
  - Eleman yoksa döngü I = N + 1 olunca biter. Eleman son gözdeyse döngü I = N iken eşitlik yüzünden biter ve N yazdırılır.
  - Eleman bir kez aranıyorsa while kullanılır. Aynı eleman birden fazla geçiyorsa ve hepsinin yeri isteniyorsa bütün diziye bakmak gerekir, o zaman for kullanılır.
- Sıralı dizide arama (search on sorted array)
  - Aynı algoritma sıralı dizide de çalışır. Ama dizi sıralı olduğu için daha iyisi yapılabilir: aranan elemandan büyük bir elemana gelindiğinde aramaya devam etmeye gerek yoktur.
  - Doğru çözüm: `I = 1`. While: `I < N AND X > Dizi[I]` olduğu sürece `I = I + 1`.
  - Döngüden sonra `X = Dizi[I]` ise I yazdırılır, değilse “Eleman bulunamadı” yazdırılır.
  - Örnek (`5 8 20 32 45` içinde 9 aranıyor): 5 ve 8 kontrol edilir. 20 > 9 olduğu için arama durur. Sırasız dizi algoritması ise dizinin sonuna kadar giderdi.
  - Bu yöntemle en kötü durumda N adımda arama tamamlanır. Dizinin ortasına bakıp büyük/küçük durumuna göre ilerleyen ikili arama (binary search) çok daha az adımda sonuç verir. Zaman kalırsa dönem sonunda anlatılacak.
- Ekleme (insert operation) (ekrandan, 01:20:30)
  - Sırasız dizide en ucuz yol, yeni elemanı son elemandan sonraki göze, yani `Dizi[N+1]`’e yazmaktır. Başa eklemek bütün elemanları kaydırmayı gerektirir.
  - Sıralı dizide eleman, sırayı bozmayacak yere yerleştirilmelidir. Örnek: `1 3 5 7 9 11` dizisine 10 eklenince `1 3 5 7 9 10 11` olur.
  - Akış diyagramı:
    - N, `Dizi[N]` ve X okunur. `I = 1`.
    - While: `I <= N AND X > Dizi[I]` olduğu sürece `I = I + 1`. Döngü bitince I, X’ten büyük ilk elemanın yeridir.
    - For `J ← N || −1 || I`: `Dizi[J+1] = Dizi[J]`. Elemanlar sondan başlanarak bir sağa kaydırılır.
    - `Dizi[I] = X`. Dizinin boyutu bir arttığı için `N = N + 1`.
  - Kaydırma sondan başlamalıdır. Baştan başlanırsa (ör. 7’yi 9’un üzerine yazmak) sağ-

daki eleman kaybolur.
  - Uç durumlar: en büyük eleman eklenirken (ör. 12) for döngüsüne hiç girilmez ve eleman `Dizi[N+1]`'e yazılır. En küçük eleman eklenirken (ör. 0) bütün dizi kaydırılır ve eleman ilk göze yazılır. İki durumda da ek bir kontrol gerekmez.
- Silme (delete operation) (öğrencinin çizimi ekrandan, 02:04:20)
  - Sırasız dizide (`1 4 5 3 2 8 9`, 2 siliniyor): silinen elemanın sağındaki elemanlar bir sola kaydırılır. Kaydırma silinen elemanın yanındaki elemandan başlar ve dizinin sonuna doğru gider. Sonuç: `1 4 5 3 8 9`.
  - Sonda boşalan göze 0 veya −1 gibi bir değer yazılabilir. N mutlaka bir azaltılmalıdır.
  - Sıralı dizide silme: silinecek elemanı ararken, aranan elemandan büyük bir elemana gelindiğinde arama durdurulur. Sırasız dizide ise bütün dizi gezilmelidir.
  - Akış diyagramı (sıralı dizi):
    * N, `Dizi[N]` ve X okunur. `I = 1`.
    * While: `I < N AND X > Dizi[I]` olduğu sürece `I = I + 1`.
    * `X = Dizi[I]` ise for `J ← I || 1 || N−1`: `Dizi[J] = Dizi[J+1]`, sonra `N = N − 1`. Değilse eleman bulunamadı.
- Bubble sort (ekrandan, 02:34:40)
  - Mantık: yan yana iki değer karşılaştırılır. Soldaki büyükse yerleri değiştirilir.
  - Her tur, kalan elemanların en büyüğünü en sona taşır. Örnek: `5 1 3 2 4 7 8 6` dizisinde ilk turdan sonra 8 en sondadır.
  - Akış diyagramı:
    * N ve `Dizi[N]` okunur.
    * Dış döngü `I ← 1 || 1 || N−1`, iç döngü `J ← 1 || 1 || N−1`.
    * `Dizi[J] > Dizi[J+1]` ise yer değiştirme (swap): `tmp = Dizi[J]`, `Dizi[J] = Dizi[J+1]`, `Dizi[J+1] = tmp`.
  - N eleman için N − 1 tur yeterlidir. 8 elemanlı dizide 7. turda 2 yerine oturduğunda 1 de kendiliğinden yerindedir.
  - İyileştirme: her turda sondaki bir eleman yerine oturduğu için iç döngünün sonu `N−1` yerine `N−I` yapılabilir. Böylece yerine oturmuş elemanlar boşuna karşılaştırılmaz.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınır değerleri çok önemlidir. Sıralı dizi aramasında `I <= N` yazılırsa, aranan eleman dizideki her elemandan büyük olduğunda döngü I = N + 1'de biter ve `Dizi[N+1]` okunur. Bu göz diziye ait değildir. `I < N` yazıldığında bu sorun ortadan kalkar.
- Karşılaştırmanın yönü de önemlidir. Sıralı dizide döngü `X > Dizi[I]` olduğu sürece devam etmelidir. `X < Dizi[I]` yazılırsa döngüye hiç girilmeyebilir.
- Bu tür çözümleri ezberlemeyin. Önce kağıt üzerinde elle nasıl yapacağınızı düşünün, sonra akış diyagramına çevirin. Analiz yapmadan "oldu" deyip geçmeyin.
- Silme işleminde aranan eleman dizide olmayabilir. While'dan çıktıktan sonra `X = Dizi[I]` kontrolü yapılmazsa, olmayan bir eleman için kaydırma yapılır ve yanlış bir eleman silinir.
- Silme döngüsünde `X ≠ Dizi[I]` koşulu da doğru çalışır. Ama sıralı dizide aranan elemandan büyük elemanlar da gereksiz yere kontrol edilir.
- Eleman eklendiğinde veya silindiğinde dizinin boyutunu tutan N güncellenmelidir. Ekleme yapılınca `N = N + 1` for döngüsünden sonra yazılmalıdır. While'dan hemen sonra yazılırsa kaydırma bir göz fazla başlar.
- Bir döngüde bulunan konum (I), başka bir döngünün başlangıç değeri olarak kullanılabilir. Silme işleminde J, I'dan başlar.
- Sıralı dizide tekrar eden elemanlar varsa ekleme döngüsünde `X > Dizi[I]` yerine `X >= Dizi[I]` kullanmak, yeni elemanı tekrar edenlerin sonuna koyar ve kaydırma sayısını azaltır.
- Dizi gözüne her erişim, yani bellekten okuma ve belleğe yazma bir maliyettir. CPU ile bellek arasındaki bağlantı, CPU'nun kendi içindeki veri aktarımından her zaman daha yavaştır. Bu yüzden ardışık dizilerde (sequential array) ekleme ve silme maliyetlidir.
- Dizi okuma için `N, Dizi[N]` kısaltması derste yalnızca zaman kazanmak için kullanıldı. Aksi söylenmedikçe ödev ve sınavlarda okuma, döngüyle tek tek yapılmaya devam edecek.
- Süre verilen alıştırmaları küçük bir sınav gibi değerlendirin. Bu sürede kendi başınıza bir yere gelebiliyorsanız ödevde ve sınavda da sorun çıkmaz.
- Bu algoritmaların kodlarını yazıp farklı durumlar için çalıştırmanız şiddetle tavsiye edilir.

Kısa Tekrar Notları

- Dört temel işlem: search, insert, delete, update. Update, search ile bulunan yerde değeri değiştirmektir.
- Sırasız arama: `I <= N AND Dizi[I] ≠ X`, çıkışta `I <= N` ise bulundu.
- Sıralı arama: `I < N AND X > Dizi[I]`, çıkışta `X = Dizi[I]` ise bulundu.
- Doğrusal aramanın en kötü durumu N adımdır. Binary search çok daha az adım gerektirir.
- Sırasız diziye ekleme: `Dizi[N+1] = X`.
- Sıralı diziye ekleme: yeri bul, sondan I'ya kadar sağa kaydır, `Dizi[I] = X`, `N = N + 1`.
- Sıralı diziden silme: yeri bul, varsa I'dan $N-1$'e kadar sola kaydır, `N = N - 1`.
- Kaydırmadan silme: silinen göz 0 veya -1 ile işaretlenir. Kaydırma maliyeti olmaz ama göz yer kaplamaya devam eder.
- Swap: `tmp = a`, `a = b`, `b = tmp`.
- Bubble sort: $N - 1$ tur, her turda en büyük eleman sona gider. İç döngü sınırı `N-I` olabilir.

Detaylı Açıklamalar

Derste dizilerdeki dört temel işlem (arama, ekleme, silme, güncelleme) sıralı ve sırasız diziler için ayrı ayrı ele alınır. Ardından sıralama problemine geçilir ve bubble sort anlatılır. Bir sonraki derste bubble sort tekrar edilecek, iyileştirilmiş hâli konuşulacak ve selection sort'a geçilecek.

Sıralı dizide arama: sınır değerinin bulunması

Sıralı dizide aramanın ilk denemesinde döngü koşulu `X < Dizi[I]` olarak yazıldı. `5 8 20 32 45` dizisinde 9 arandığında $9 < 5$ yanlış olduğu için döngüye hiç girilmedi. Koşulun `X > Dizi[I]` olması gerektiği görüldü.

İkinci denemede koşul `I <= N AND X > Dizi[I]` idi. `1 2 3 4 5 6 8` dizisinde (N = 7) 9 arandığında:

- I = 7'de $9 > 8$ olduğu için I = 8 olur.
- $8 \leq 7$ yanlış olduğu için döngüden çıkılır.
- Ardından `X = Dizi[8]` kontrol edilir. `Dizi[8]` diziye ait olmayan bir gözdür.

Önerilen düzeltmeler:

- Döngüden sonra önce `I <= N` kontrol etmek. Çalışır, ama ek bir kontrol gerekir.
- `Dizi[I-1]` ile karşılaştırmak veya I'yı bir azaltmak. Bu, 8 gibi döngü içinde bulunan elemanlarda yanlış sonuç verir.
- Döngü koşulunu `I < N` yapmak. Seçilen çözüm budur.

`I < N` ile 8 arandığında döngü I = 7'de biter ve `Dizi[7] = 8` bulunur. 9 arandığında da döngü I = 7'de biter. `Dizi[7] = 8 ≠ 9` olduğu için "Eleman bulunamadı" yazdırılır. Döngüden sonra `I <= N` sorusu yeterli değildir. Koşul `X > Dizi[I]` olduğu için döngüden hem eşitlik hem de küçüklük yüzünden çıkılmış olabilir.

Bir öğrenci iki algoritmanın görünüşte aynı olduğunu, neden ikincisinin yazıldığını sordu. Fark şudur: sıralı dizi algoritması, aranan elemandan büyük bir elemana geldiğinde durur. `5 8 20 32 45` içinde 9 aranırken yalnızca 5, 8 ve 20 kontrol edilir. Sırasız dizi algoritması dizinin sonuna kadar gider.

Sıralı diziye ekleme: adım adım

`1 3 5 7 9 10 11` dizisine (N = 7) 6 eklenirken:

1. While döngüsü $6 > 1$, $6 > 3$, $6 > 5$ için devam eder. $6 > 7$ yanlış olduğu için I = 4'te durur.
2. For döngüsü J = 7'den 4'e kadar geriye doğru gider: 11 sekizinci göze, 10 yedinci göze, 9 altıncı göze, 7 beşinci göze kopyalanır.
3. `Dizi[4] = 6` yazılır. Sonuç: `1 3 5 6 7 9 10 11`.
4. `N = N + 1` ile N = 8 olur.

N'nin güncellenmesi gerekir, çünkü dizinin boyutunu temsil eden değişken N'dir. Sonradan dizi üzerinde başka bir iş yapılacaksa N'nin yeni eleman sayısını göstermesi gerekir. Dizi tanımlanırken 100, 200 gibi büyük bir boyut verildiği için `N+1`'inci göz zaten kullanılabilir. Kullanıcının verdiği sayıya göre yer ayrılsaydı bellekte yeniden yer ayırmak gerekirdi. Bu C'de ve ileriki derslerde görülecek.

Bir diziye başka bir dizinin elemanları eklenecekse, bu işlem o dizideki eleman sayısı kadar tekrarlanır. Bunu daha akıllıca yapmanın yöntemleri de vardır.

Kaydırmanın maliyeti ve alternatifler

Ekleme ve silmede kaydırma işlemi dizinin pek çok gözüne erişmeyi gerektirir. Atama işlemi basit görünse de her erişim bellekten okuma ve belleğe yazma demektir. Veriler önbellekte (cache) değilse bu işlem karşılaştırma ve dört işlem kadar maliyetlidir.

Kaydırmadan kaçınmanın yolları:

- Bağlı liste (linked list): veriler belleğin farklı bölgelerinde saklanır ve birbirine adreslerle bağlanır. Yeni eleman, diğer elemanlara dokunmadan iki elemanın arasına yerleştirilir. Bu yapı ileride detaylı görülecek.
- İşaretleyerek silme: silinen göze 0 veya -1 gibi bir değer yazılır. Kaydırma yapılmaz, ama göz yer kaplamaya devam eder (alan karmaşıklığı düşmez).
- Silinen gözlerin yerleri başka bir dizide tutulabilir. Okuma sırasında bu gözler atlanır.
- Boşalan gözler yeni elemanlar için yeniden kullanılabilir. Bilgisayarın belleği de böyle çalışır: bir programın işi bitince bellekteki alanın boşaldığı bilgisi işletim sisteminde tutulur. Eski veri orada kalır, ama alan başka bir programa verilebilir.

Bellek biçimlendirmede de benzer bir ayrım vardır. Hoca bunu "hard reset" ve "soft reset" olarak adlandırdı. Birincisinde bütün gözlerin üzerine yazılır ve uzun sürer. İkincisinde değerler yerinde kalır, yalnızca alanın kullanıma açıldığı bilgisi saklanır.

Silme algoritmasının düzeltilmesi

Bir öğrencinin gönderdiği çizimde while koşulu `I <= N AND X > dizi[I]` idi. Ardından doğrudan for `J ← I || 1 || N−1` ile `dizi[J] = dizi[J+1]` yapılıyor ve `N = N − 1` ile bitiriliyordu. For'un $N-1$'e kadar gitmesi doğrudur, çünkü atama `J+1`'den yapılır. N'e kadar gidilseydi `dizi[N+1]`'e erişilirdi.

Çizimde eksik olan, aranan elemanın dizide olmayabileceğidir. `1 3 5 7 8 9` dizisinden 6 silinmek istendiğinde döngü 7'de durur. Kontrol olmadığı için 7 silinir. Bu yüzden while ile for arasına `X = Dizi[I]` kontrolü konur. Eleman bulunamazsa false koluna gidilir ve kaydırma yapılmaz.

Bu kontrolün doğru çalışması için while koşulu da `I < N` yapılmalıdır. 11 silinmek istendiğinde `I <= N` ile döngü I = 7'de (N = 6) biter ve `Dizi[7]` ile karşılaştırma yapılır.

Tekrar eden elemanların hepsi silinmek istenirse (ör. `1 3 5 5 5 7`), önce kaç tekrar olduğu bulunur. Sonra tekrarların sağındaki elemanlar, tekrar sayısı kadar sola kaydırılır. Hoca bu varyasyonu ve benzerlerini (birden fazla eleman ekleme, silme) alıştırma olarak önerdi.

Bubble sort: döngü sınırları

`5 1 3 2 4 7 8 6` dizisinin ilk turu:

- 5 ile 1 yer değiştirir: `1 5 3 2 4 7 8 6`.
- 5 ile 3, 5 ile 2, 5 ile 4 yer değiştirir: `1 3 2 4 5 7 8 6`.
- 5 ile 7 ve 7 ile 8 yerinde kalır.
- 8 ile 6 yer değiştirir: `1 3 2 4 5 7 6 8`.

İlk turda en büyük eleman (8) sona yerleşir. İkinci turda 7, üçüncü turda 6 yerleşir. Karşılaştırmaya her turda baştan başlanmalıdır. Örneğin 1 dizinin sonunda olsaydı, ancak her turda bir sola kayarak başa ulaşabilirdi.

İç döngü 1'den N−1'e gider, çünkü son karşılaştırma `Dizi[N−1]` ile `Dizi[N]` arasındadır. Dış döngü de N−1 kez döner. Bir öğrencinin önerisiyle iç döngünün sonu `N−I` yapılır. I. turda sondaki I−1 eleman zaten yerine oturmuştur ve onların tekrar karşılaştırılmasına gerek yoktur.

İki elemanın yerini değiştirmek için geçici bir değişken (`tmp`) gerekir. 1'i 5'in üzerine yazarsanız 5 kaybolur, 5'i 1'in üzerine yazarsanız 1 kaybolur.

Hoca, bubble sort'un daha akıllıca hâle nasıl getirilebileceğini düşünmeyi öğrencilere bıraktı. Ders sonrasında yapılan kontrolde arama, ekleme, silme ve bubble sort algoritmaları rastgele dizilerle denendi ve hepsi doğru sonuç verdi.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.